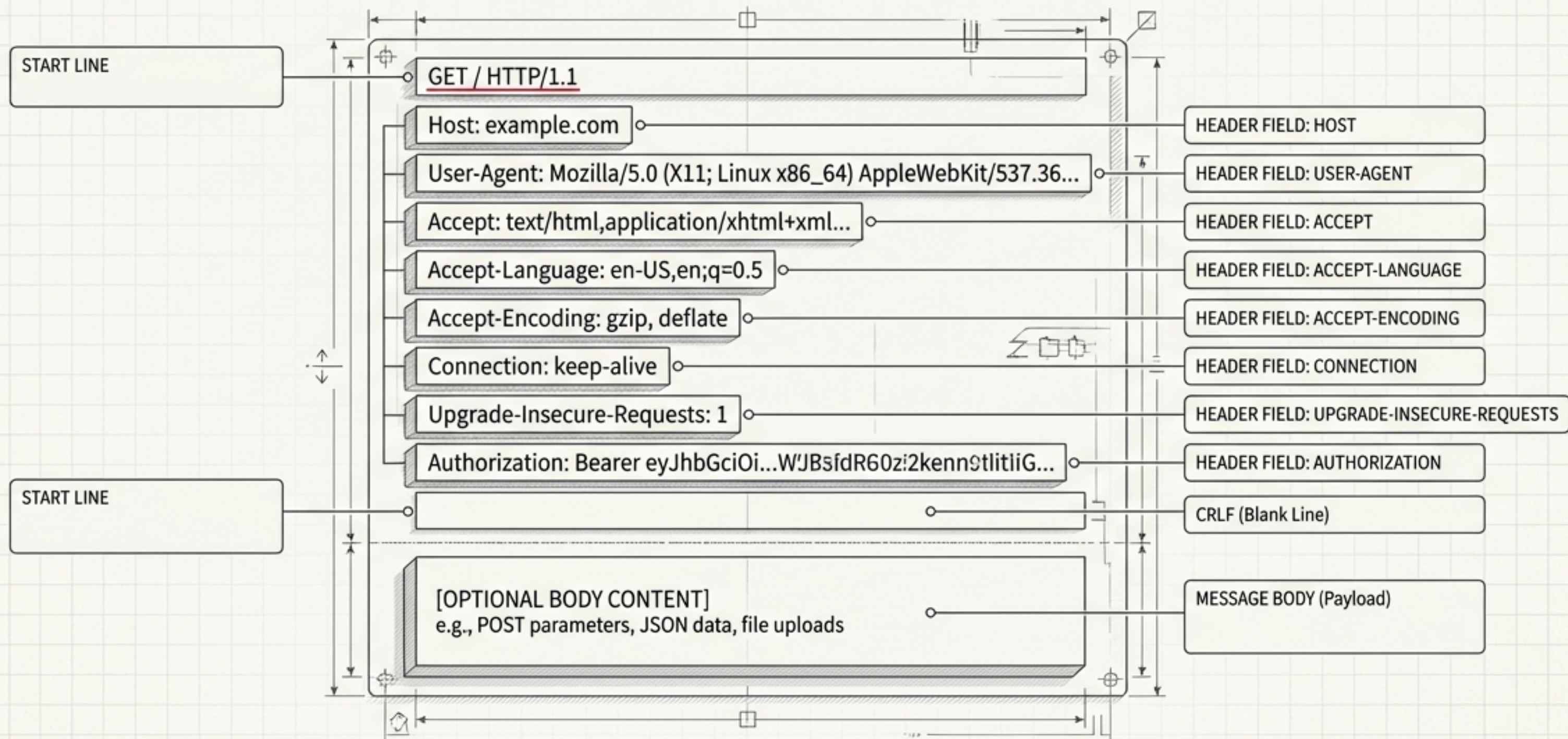


Anatomía de una Petición HTTP

Manual de Campo para el Pentester



El Lenguaje Universal de la Web

```
```http
GET /descargas/index.php HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip, deflate
Connection: keep-alive
Cookie: user_type=student;
room=introtowebapplication
```

Esto es una petición HTTP, el lenguaje que sustenta toda la web. Para un desarrollador, es una simple solicitud de datos. Para un pentester, es un **mapa de superficies de ataque**. Cada línea, cada encabezado y cada byte del cuerpo es una oportunidad para descubrir vulnerabilidades.

Comencemos la disección.



# La Línea de Solicitud: Qué, Dónde y Cómo



```http

GET /descargas/index.php **HTTP/1.1**

Host: www.google.com

User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64)

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

lication/xml;q=0.9,*/*;q=0.8

Accept-Encoding: gzip, deflate

Connection: keep-alive

Cookie: user_type=student;
room=introtowebapplication

MÉTODO:

La acción

a realizar

(`GET`,

`POST`.

• **RUTA:** El recurso específico (`/`,
`/api/users/123`).

• VERSIÓN HTTP:

Las reglas del juego
(`HTTP/1.1`,
`HTTP/2`).

La primera línea de toda petición define la intención principal. Se descompone en tres partes críticas.

La Ruta (Path): El Mapa Hacia el Recurso

La Función

GET `/descargas/index.php` HTTP/1.1

La ruta URL indica al servidor dónde encontrar el recurso solicitado. Especifica la ubicación exacta, desde la raíz (/) hasta endpoints complejos (/api/users/123).



La Perspectiva del Pentester (El Riesgo)

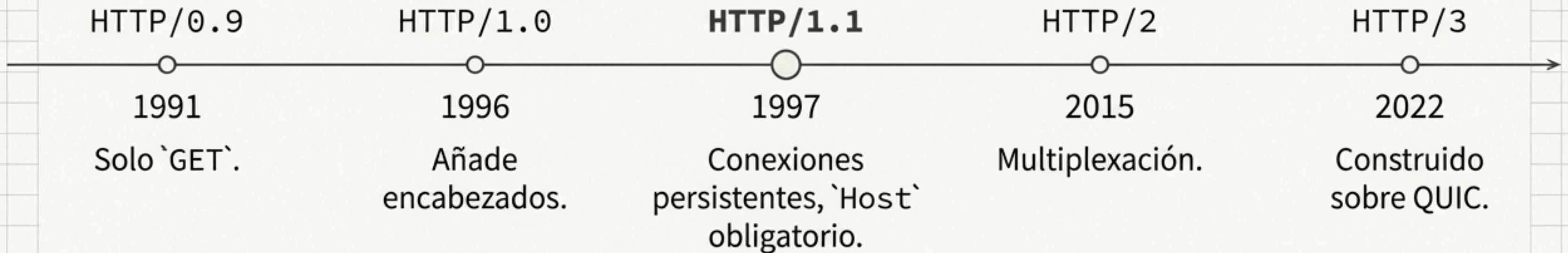
La manipulación de la ruta es un vector de ataque primario. Los errores de validación y saneamiento del servidor pueden conducir a:

- **Path Traversal:** Acceder a archivos fuera del directorio permitido (ej. .././etc/passwd).
- **Insecure Direct Object References (IDOR):** Exponer datos sensibles cambiando identificadores (ej. /api/users/123 a /api/users/124).
- **Fuzzing de Rutas:** Descubrir endpoints ocultos o de administración (/admin, /backup).



La Versión HTTP: Definiendo las Reglas del Juego

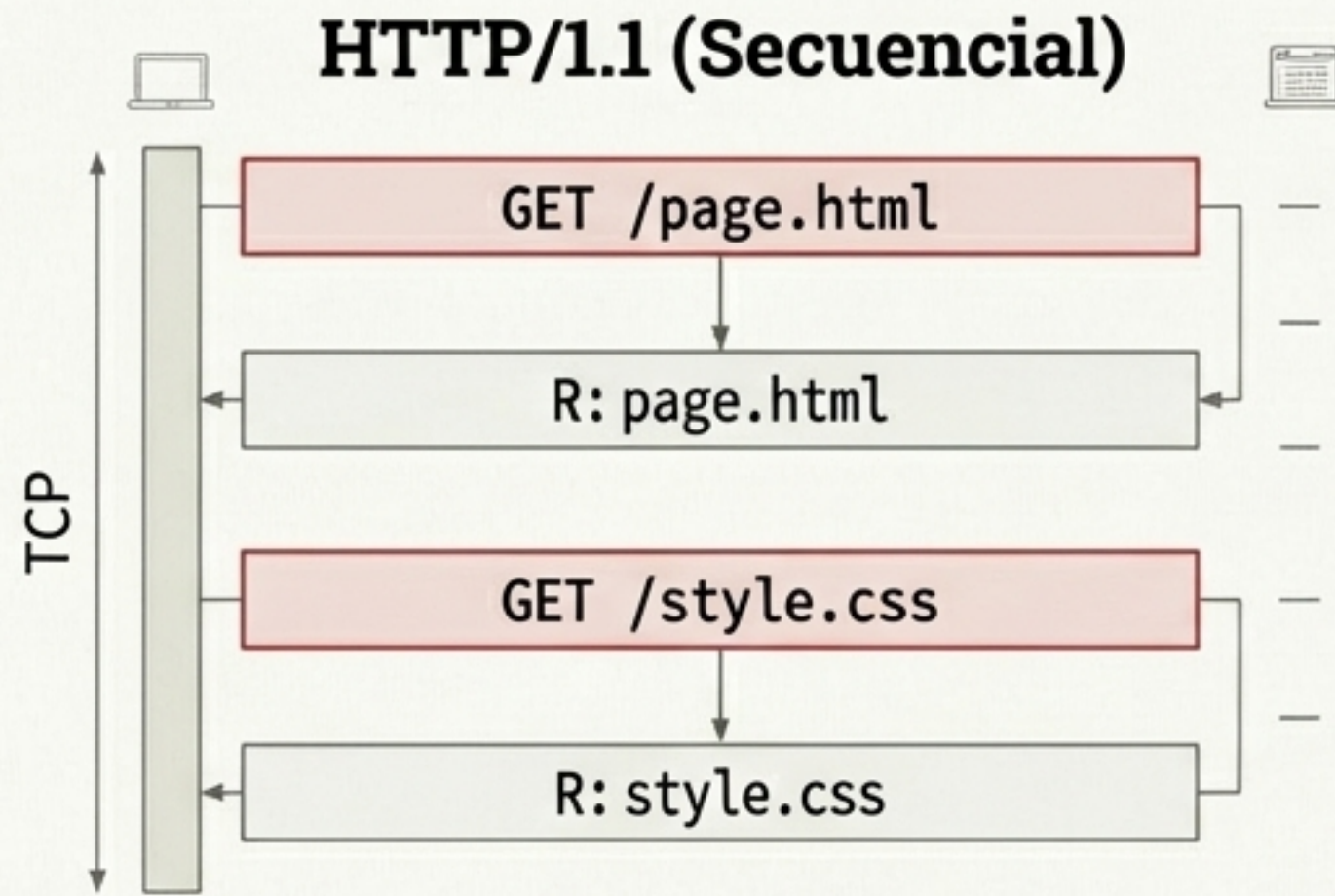
GET /descargas/index.php **HTTP/1.1**



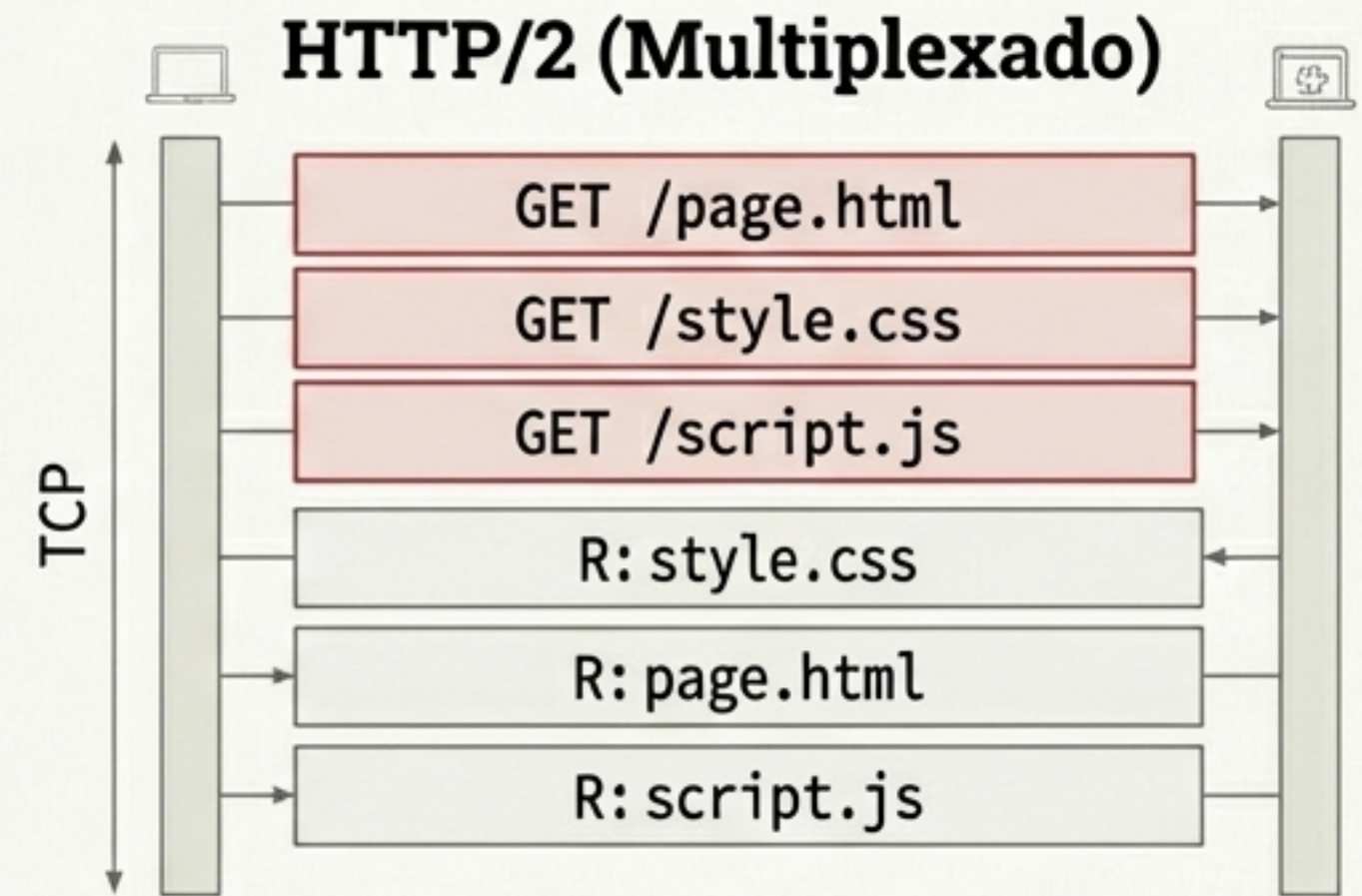
Perspectiva del Pentester

- La versión puede delatar sistemas antiguos o mal configurados.
- Entender las diferencias entre versiones es clave para detectar vulnerabilidades como **HTTP Request Smuggling**, que explotan inconsistencias de parsing entre proxies y servidores.

El Salto a la Eficiencia: HTTP/1.1 vs. HTTP/2



Cada recurso se solicita de forma secuencial.
Genera latencia y 'head-of-line blocking'.



Múltiples peticiones y respuestas viajan en paralelo sobre una sola conexión, optimizando drásticamente la velocidad.

Nota de Seguridad: Esta complejidad puede ocultar patrones de ataque tradicionales. Es crucial probar las aplicaciones en ambos protocolos para detectar diferencias de comportamiento.

Los Encabezados: El Contexto Crítico de la Petición

Debajo de la línea de solicitud, los encabezados (`headers`) proporcionan información adicional que guía la interacción. Siguen una estructura simple **Nombre-Encabezado: Valor** y son una fuente inagotable de vulnerabilidades.

```
GET /descargas/index.php HTTP/1.1
```

```
Host: www.google.com
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110
```

```
Safari/537.36
```

```
Accept: text/html,application/xhtml+xml,application/xml;  
q=0.9,image/webp,*/*;q=0.8
```

```
Accept-Encoding: gzip, deflate, sdch
```

```
Accept-Language: es-ES,es;q=0.8
```

```
Connection: keep-alive
```

```
Cookie: NID=101=t55...
```

Host: El dominio de destino.

User-Agent: La identidad del cliente.

Accept / Accept-Encoding:
Las capacidades del cliente.

Connection: El estado de la conexión.

Cookie: El estado de la sesión.

El Encabezado `Host`: ¿A Quién le Estás Hablando?

La Función

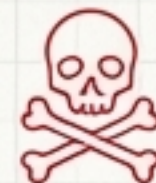
```
GET / HTTP/1.1
User-Agent: ...
Accept: ...
Host: www.google.com
Accept: www.google
Connection: 119
Reader: coder/vank
Hosts: www.google.
...
```

Obligatorio en HTTP/1.1.

Permite que un solo servidor web aloje múltiples sitios web en una misma dirección IP (virtual hosting).

Especifica el dominio al que va dirigida la petición.

El Riesgo: Host Header Injection



Si una aplicación confía ciegamente en este valor para generar URLs, puede ser explotada.

Vectores de Ataque:

- Envenenamiento de Caché Web
- Envenenamiento de Restablecimiento de Contraseña
- Redirecciones Maliciosas

****Defensa Clave**:** Validar siempre el valor del `Host` contra una lista blanca de dominios permitidos.

El `User-Agent`: La Identidad Maleable del Cliente

La Función

```
GET / HTTP/1.1
User-Agent: ...
User-Agent: Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/58.0.3029.110 Safari/537.36
Connection: 119
Reader: coder/vank
Hosts: www.google.
...
```

Identifica el software que realiza la solicitud (navegador, versión, SO). El servidor lo usa para adaptar el contenido (ej. versión móvil vs. escritorio).

La Perspectiva del Pentester

Este encabezado es trivial de falsificar. Un pentester lo manipula para:

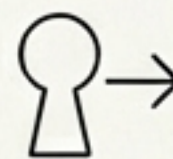
Tácticas de Manipulación:



Evadir WAFs y Filtros: Simular ser un bot legítimo (Googlebot) o un navegador común para evitar bloqueos.



Fingerprinting de Servidor: Observar cómo cambian las respuestas a diferentes User-Agents para identificar frameworks o versiones de CMS.



Descubrir Lógica Oculta: Acceder a endpoints o funcionalidades reservadas para clientes específicos (ej. aplicaciones móviles).

Negociación de Contenido: Pidiendo Más de la Cuenta

GET / HTTP/1.1

User-Agent: ...

Accept: application/json, text/html;q=0.9, */*;q=0.8

Accept-Encoding: gzip, deflate, br

...

`Accept`

(Función): Comunica qué tipos de contenido (MIME types) acepta el cliente (ej. `text/html`, `application/json`).

(Riesgo): Cambiar `Accept: application/json` en un endpoint que normalmente devuelve HTML puede **exponer una API interna no documentada** o provocar rutas de error que filtren información.

`Accept-Encoding`

(Función): Indica los algoritmos de compresión soportados (ej. `gzip`, `deflate`) para ahorrar ancho de banda.

(Riesgo): La compresión puede introducir vectores sutiles de **fuga de información lateral** en respuestas cifradas (ataques como BREACH). También, inconsistencias en el manejo de la compresión pueden delatar una mala configuración.

El Encabezado `Connection`: Controlando el Flujo

La Función

```
GET / HTTP/1.1
User-Agent: ...
...
Connection: keep-alive
...
Connection: 119
Reader: coder/wank
Hosts: www.google.
...
```

Gestiona la persistencia de la conexión TCP. Con `Connection: keep-alive`, la misma conexión se reutiliza para múltiples peticiones, reduciendo la latencia.

El Riesgo: Request Smuggling



Este encabezado es una pieza clave en ataques de **HTTP Request Smuggling**. La vulnerabilidad surge cuando un proxy y el servidor final interpretan de manera diferente los límites de una petición (ej. `Content-Length` vs. `Transfer-Encoding`).

Un atacante puede 'contrabandear' una segunda petición maliciosa dentro del cuerpo de una petición aparentemente inocente, saltándose los controles de seguridad del proxy.

El Cuerpo de la Petición: La Carga Útil

En peticiones `POST` o `PUT`, el cuerpo contiene los datos enviados al servidor. El formato varía, pero la mayoría de las vulnerabilidades de inyección residen aquí.

URL Encoded (`application/x-www-form-urlencoded`)

```
key1=value1&key2=value2
```

JSON (`application/json`)

```
{  
  "nombre": "Aleksandra",  
  "rol": "pentester"  
}
```

XML (`application/xml`)

```
<user>  
  <name>Aleksandra</name>  
  <role>pentester</role>  
</user>
```

Form Data (`multipart/form-data`)

Usado principalmente para subir archivos.
Cada parte está separada por un “boundary”.

Riesgo Principal:** Este es el campo de batalla para **SQL Injection (SQLi)**, **Cross-Site Scripting (XSS)**, **XML External Entity (XXE)** y fallos de lógica de negocio.

La Anatomía Completa: El Mapa de Ataque del Pentester

Comando: Define la intención. Riesgo de **Path Traversal**, **IDOR**.

Identidad: Describe al cliente. Manipulable para evadir WAFs.

Payload: Los datos enviados. Vector principal para **SQLi**, **XSS**, **XXE**.

```
GET /descargas/index.php HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124
Safari/537.36
Cookie: user_type=student; session_id=1234567890abcdef

{
  "search_term": "query"
}
```

Destino: Especifica el servidor. Riesgo de **Host Header Injection**.

Estado: Mantiene la sesión. Objetivo para **Session Hijacking**.

Separador `\r\n`: La frontera crítica. Punto clave para **Request Smuggling**.

Cada Línea es un Vector

Hemos disecado la petición HTTP, revelando que no hay partes benignas. Cada línea es un vector potencial. Cada valor es una variable que puede ser manipulada.

Comando: Define la intención. Riesgo de Path Traversal.

Identidad: Describe al cliente. Manipulable para evadir WAFs.

Estado: Mantiene la sesión. Riesgo de Session Hijacking.

Payload: Los datos enviados. Vector principal para SQLi, XSS, XXE.

Separador: La frontera crítica. Punto clave para Request Smuggling.

```
GET /descargas/index.php HTTP/1.1
Host: 10.10.10.10
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/123.0.0.0 Safari/537.36
Cookie: user_type=1234567890abcdef
```

Entender esta anatomía no es solo conocimiento técnico; es la habilidad fundamental de un pentester.

AHORA, EXPLORA. MANIPULA. ASEGURA.